

Agile

Why Agile?

At first glance, Agile may seem like additional overhead with daily standups, sprint ceremonies, and frequent planning sessions, it can feel like more meetings and process than traditional development. However, the true value of Agile lies not in the rituals themselves, but in what they enable:

Skill Development and Alignment

Agile creates a structured environment where individuals are constantly practicing core competencies: **collaboration**, **communication**, iterative delivery, problem-solving, and self-organization.

- **For individuals:** It fosters continuous learning through regular feedback, pair programming, retrospectives, and cross-functional teamwork.
- **For companies:** It offers a clear system for identifying and developing talent teams can easily recognize who has the mindset and skills to thrive in dynamic, team-driven environments.

Integrated with Workflows

Agile practices are designed to fit seamlessly into modern development and business workflows.

- Breaks down silos between engineering, design, product, and QA
- Supports CI/CD pipelines, DevOps, and modern tooling, MLops
- Encourages fast feedback loops and regular customer input

This integration ensures that work is delivered not only faster, but more aligned with real user needs.

Common Language Across Teams

Agile provides a shared vocabulary that makes collaboration easier, especially in cross-functional and distributed teams.

- Terms like "sprint," "user story," "Definition of Done," and "velocity" mean the same thing to everyone

- Reduces miscommunication and increases clarity
- Enables better collaboration between technical and non-technical stakeholders

Adaptability and Value Delivery

Agile is built for change. In fast-moving markets, customer needs and priorities evolve constantly.

- Agile teams respond quickly to change, rather than resisting it
- Work is delivered incrementally, allowing for faster feedback and course correction
- Focus remains on delivering value not just output

Core Principles

Agile Manifesto Values:

- Individuals and interactions over processes and tools
- Working software over comprehensive documentation
- Customer collaboration over contract negotiation
- Responding to change over following a plan

Key Principles:

- **Deliver working software frequently** (every couple of weeks to a couple of months)
- Welcome changing requirements, even late in development
- Business people and developers must work together daily
- Build projects around motivated individuals
- Face-to-face conversation is the most efficient communication method
- Working software is the primary measure of progress
- Maintain a sustainable pace of development
- Continuous attention to technical excellence and good design

- Simplicity – maximize the amount of work not done
 - Self-organizing teams produce the best results
 - Regular team reflection and adjustment of behavior
-

Scrum Framework

Roles

Product Owner

- Defines product vision and roadmap
- Manages and prioritizes product backlog
- Writes user stories and acceptance criteria
- Makes decisions on scope and timeline
- Acts as customer representative

Scrum Master

- Facilitates Scrum events and removes impediments
- Coaches the team on Agile practices
- Protects the team from external distractions
- Ensures adherence to the Scrum framework
- Promotes continuous improvement

Developers

- Cross-functional professionals on the Scrum Team
- Self-organizing and accountable for deliverables
- Estimate effort for backlog items
- Commit to sprint goals
- Deliver usable increments of value

Events (Ceremonies)

Sprint Planning (max 8 hours for a one-month sprint; shorter for shorter sprints)

- Select backlog items for the upcoming sprint
- Define sprint goal and commitment
- Break down stories into tasks
- Estimate effort and capacity

Daily Scrum (15 minutes)

- What did I accomplish yesterday?
- What will I work on today?
- What obstacles are blocking my progress?

Sprint Review (max 4 hours for a one-month sprint)

- Demo completed work to stakeholders
- Gather feedback on the product increment
- Discuss what was and wasn't accomplished
- Review and update the product backlog

Sprint Retrospective (max 3 hours for a one-month sprint)

- What went well during the sprint?
- What could be improved?
- What will we commit to improve in the next sprint?
- Action items for team improvement

Artifacts

Product Backlog

- Prioritized list of features and requirements
- Written as user stories with acceptance criteria
- Continuously refined and updated
- Estimated using story points or other methods

Sprint Backlog

- Items selected for the current sprint
- Tasks broken down from user stories

- Updated daily by the Developers
- Includes the sprint goal

Increment

- Sum of all completed backlog items during the sprint
- Must be potentially releasable
- Meets the Definition of "Done"
- Cumulative across all sprints

Work Item Types

Sprint

Time-boxed iteration where a team commits to completing a set of work items

- Timeline: Fixed duration (typically 1-4 weeks, most commonly 2 weeks)
- Size: Contains multiple stories, bugs, and tasks that can be completed within the sprint timeframe
- Example: "Sprint 15: User Authentication Features (March 1-14)"
- Contains: Stories, bugs, tasks selected from the product backlog
- Goal: Deliver a potentially shippable product increment

Epic

Large body of work that spans multiple sprints and can be broken down into smaller stories

- Timeline: Multiple sprints (weeks to months)
- Size: Too large to complete in one sprint
- Example: "Implement user authentication system"
- Contains: Multiple user stories and potentially bugs/tasks

If work can be completed within a single sprint, it should be a Story, not an Epic.

Story (User Story)

Small, independent piece of functionality from the user's perspective

- Timeline: Completable within one sprint
- Size: 1-13 story points typically
- Example: "As a user, I want to reset my password so I can regain access to my account"
- Contains: Tasks and potentially sub-tasks

Bug

Defect or issue that needs to be fixed

- Timeline: Varies based on severity (critical bugs = immediate, minor bugs = next sprint)
- Size: Usually small to medium effort
- Example: "Login button doesn't work on mobile Safari"
- Contains: Tasks for investigation and fixing

Task

Specific work item that doesn't deliver user value directly but supports story completion

- Timeline: Hours to days
- Size: Small, concrete work items
- Example: "Set up database table for user preferences"
- Contains: May have sub-tasks for complex work

Sub-task

Smallest unit of work, part of a larger task or story

- Timeline: Hours
- Size: Very small, specific actions
- Example: "Write unit tests for password validation function"

- Contains: Nothing (atomic work item)

Hierarchy Structure

```
Epic: User Account Management
├── Story: User Registration
│   ├── Task: Create registration form
│   │   ├── Sub-task: Design form layout
│   │   └── Sub-task: Implement form validation
│   └── Task: Set up user database
├── Story: Password Reset
└── Bug: Registration email not sending
    └── Task: Debug email service configuration
```

User Stories

Format

As a **[type of user]**,

I want **[some goal or objective]**,

So that **[some reason or benefit]**.

INVEST Criteria

- **Independent:** Can be developed in any order
- **Negotiable:** Details can be discussed and changed
- **Valuable:** Provides value to end users or business
- **Estimable:** Team can estimate effort required
- **Small:** Can be completed within one sprint
- **Testable:** Clear acceptance criteria exist

Acceptance Criteria

- Define conditions that must be met for story completion
 - Written in "Given-When-Then" format when possible
 - Specific, measurable, and testable
 - Agreed upon by the Product Owner and team
-

Estimation Techniques

Story Points

- Relative sizing method using the Fibonacci sequence (1, 2, 3, 5, 8, 13, 21)
- Based on complexity, effort, and uncertainty
- Team-specific and improves over time
- Used for sprint planning and velocity tracking

Planning Poker

- Team estimates stories collectively
- Each member selects an estimate privately
- Discuss differences and re-estimate
- Continues until consensus is reached

T-Shirt Sizing

- High-level estimation using XS, S, M, L, XL
 - Useful for initial backlog refinement
 - Less precise than story points
 - Good for epic-level planning
-

Definition of Done

Create a team agreement on completion criteria:

- Code is written and peer-reviewed

- Unit tests are written and passing
 - Integration tests are passing
 - Code is merged to the main branch
 - Documentation is updated
 - Feature is deployed to staging environment
 - Product Owner has accepted the work
 - No critical bugs remain
 - Meets non-functional requirements (performance, security, etc.)
-

Common Metrics

Velocity

- Amount of work completed per sprint
- Measured in story points or completed stories
- Used for sprint planning and forecasting
- **Not meant for comparison across teams**
- Track trend over multiple sprints

Burndown Charts

- Visual representation of work remaining vs. time
- Sprint burndown shows daily progress
- Release burndown shows progress toward release
- Helps identify scope creep or delivery risks

Burn-Up Charts (*Optional*)

- Alternative to burndown that shows work completed vs. total scope
- Useful when scope is expected to change

Lead Time and Cycle Time

- **Lead Time:** Time from request to delivery

- **Cycle Time:** Time from start of work to completion
 - Measure flow efficiency and identify bottlenecks
 - Shorter times indicate better Agile maturity
-

Best Practices

Sprint Management

- Keep sprints short (1–4 weeks, typically 2 weeks)
- Don't change sprint scope once committed
- Focus on completing stories rather than starting new ones
- Maintain consistent sprint length

Backlog Management

- Refine backlog regularly (use ~10% of team capacity)
- Keep top items ready for the next 2–3 sprints
- Write detailed user stories with clear acceptance criteria
- Prioritize based on business value and dependencies

Team Collaboration

- Co-locate team members when possible
- Encourage face-to-face communication
- Share knowledge through pair programming
- Hold regular retrospectives and act on improvements

Technical Practices

- Implement continuous integration and deployment
 - Maintain automated test suites
 - Refactor code regularly to reduce technical debt
 - Follow coding standards and conduct code reviews
-

Common Anti-Patterns to Avoid

- **Scope Creep:** Adding work mid-sprint without removing other work
- **Mini-Waterfall:** Doing all analysis upfront, then all coding, then all testing
- **Zombie Scrum:** Going through motions without embracing Agile values
- **Feature Factory:** Focusing on output rather than outcomes and value
- **Estimation Theater:** Spending too much time on precise estimates
- **Tool Obsession:** Focusing on tools rather than individuals and interactions